

Authentication and Authorisation Architecture

Document ID: DOC-SEC-006
Classification: Official
System: Case Ace v2.0 (Citizens Advice Wandsworth)
Standard Alignment: ISO/IEC 27001:2022 (A.5.15, A.5.18, A.8.2, A.8.5), ISO/IEC 27701:2019, RFC 6238, OIDC Core 1.0
Data Scope: UK GDPR Special Category Data Access Control

1. Architectural Principles & Threat Context

Case Ace v2.0 implements strict identity and access governance tailored to a zero-persistence advice drafting system:

Mandatory Multi-Factor Authentication (MFA): No user may initiate a session without cryptographically verified multi-factor authentication.

Stateless Role-Based Access Control (RBAC): Roles are enforced server-side on every request and encoded in short-lived cryptographic tokens.

The Fundamental Privacy Guarantee:

[!IMPORTANT]

ZERO CROSS-SESSION ACCESS GUARANTEE:

No role—including supervisor, administrator, and auditor—can access another user’s consultation data, transcripts, or case notes, because no session data exists to access. Consultation data exists only in volatile RAM on the individual adviser’s device during an active interview and is destroyed upon session end.

2. The AuthProvider Interface

All authentication flows implement the unified AuthProvider interface:

```
export interface AuthProvider {
  readonly providerType: 'entra_id' | 'totp';
  authenticate(params: EntraIdAuthParams | TotpAuthParams): Promise<AuthResult>;
  verifyToken(token: string): Promise<AuthUser | null>;
  refreshToken(refreshToken: string): Promise<AuthResult>;
}
```

3. Provider Implementations

3.1 Primary Provider: EntraIdProvider (Microsoft Entra ID)

Protocol: OpenID Connect (OIDC) Authorization Code Flow with Proof Key for Code Exchange (PKCE) per RFC 7636.

MFA & Conditional Access: Enforced via CAW Entra ID Conditional Access policies requiring compliant managed devices and registered authenticator apps.

Mandatory amr (Authentication Methods References) Claim Verification:

The backend inspects the amr claim in the ID token.

The token **MUST** contain explicit MFA indicators (e.g., mfa, fido, ngcmfa, otp, hwk, swk, sms).

Fail-Closed Rule: If the amr claim is missing or only contains single-factor identifiers (e.g., ["pwd"]), the authentication attempt is immediately rejected with MFA_REQUIRED.

Role Resolution:

Security group claims (groups) drive role assignments:

grp-caw-advisers $\rightarrow$ adviser

grp-caw-supervisors $\rightarrow$ supervisor

grp-caw-admins $\rightarrow$ administrator

grp-caw-auditors $\rightarrow$ auditor

3.2 Fallback Provider: TotpProvider (RFC 6238 TOTP)

Purpose: Development, automated CI testing, and emergency operational fallback if Entra ID registration or federated

tenant connectivity stalls.

Credentials:

Username.

Password hashed with scrypt (64-byte derived key, 16-byte random salt).

6-digit Time-Based One-Time Password (TOTP) per RFC 6238 (HMAC-SHA1, 30-second time step, ± 1 step clock drift allowance).

Brute-Force Rate Limiting & Account Lockout:

5 consecutive failed authentication attempts trigger an automatic 15-minute temporary lockout (ACCOUNT_LOCKED).

Resets upon successful authentication.

Pilot Environment Prohibition:

TotpProvider is strictly forbidden in the pilot environment.

Any attempt to initialize TotpProvider under APP_ENV=pilot causes the backend to fail closed immediately at boot.

4. Configuration Mutual Exclusivity & Fail-Closed Enforcement

[!CAUTION]

FAIL-CLOSED DUAL-PROVIDER PROHIBITION:

The system configuration strictly forbids enabling both EntraIdProvider and TotpProvider simultaneously.

If configuration parameters enableEntraId and enableTotp are both evaluated as true, the factory throws an unrecoverable Security Configuration Error and halts execution.

```
export function createAuthProvider(config: AuthConfig, environmentName: string): AuthProvider {
  if (config.enableEntraId && config.enableTotp) {
    throw new Error('Security Configuration Error: Mutually exclusive auth providers cannot both be
enabled simultaneously.');
```

5. Server-Side Role-Based Access Control (RBAC) Matrix

Server-side middleware (requireAuth, requireRole) strictly gates every API endpoint. Role checks are never delegated solely to the client UI.

6. Session Lifecycle & Ephemerality

Access Token TTL: 15 minutes (900 seconds).

Refresh Token TTL / Absolute Session Cap: 8 hours (28,800 seconds).

Client Idle Inactivity Timeout:

Configured for 15 minutes of inactivity (no mouse, keyboard, or touch events).

Upon expiry, IdleTimeoutManager immediately executes:

volatileSessionStore.destroySession(): Physically zeroes all audio memory buffers (Uint8Array.fill(0)), clears token map, and wipes consultation state.

volatileAuthStore.clearAuth(): Purges access and refresh tokens from memory.

Prompts user with session expired notification.

Destructive Logout Action:

Logging out opens a mandatory modal warning the adviser that unsaved notes will be permanently erased.

Confirmation executes immediate, irreversible in-memory wiping.